

OpenCap Visualizer: A Web-Based Platform for Scriptable, Real-Time, and Interactive Visualization of Biomechanics Data

Selim Gilon¹ and Scott Uhlrich^{1,2}

¹ Department of Mechanical Engineering, University of Utah, Salt Lake City, UT, United States of America
² Department of Orthopaedic Surgery, University of Utah, Salt Lake City, UT, United States of America

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Erick Martins Ratamero](#)

Reviewers:

- [@davidpagnon](#)

Submitted: 09 April 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Biomechanics research relies on visualizing 3D movement data to interpret and validate results, but traditional desktop-based graphical user interfaces (GUIs) have become a bottleneck as the scale of biomechanics datasets grows and processing increasingly moves to cloud-based servers. Current GUIs require extensive manual interaction to load models and motions, configure scenes, and export media. To resolve these challenges, we created OpenCap Visualizer, a web-based platform and Python package that enables both interactive 3D visualization, real-time streaming, and programmatic video generation.

The software provides three primary interfaces: a browser-based viewer for shareable visualization, a WebSocket interface for real-time streaming, and a Python API for automated rendering. Built with Vue.js and Three.js, it supports standard biomechanics formats—including OpenSim models (.osim), kinematics (.mot, .json), markers (.trc), and force data (.mot)—allowing researchers to process, analyze, render, and share biomechanics videos with minimal human intervention.

The platform is available at <https://www.visualizer.opencap.ai>.

Statement of Need

Biomechanics datasets are growing by orders of magnitude due to recent advances in mobile sensing, including wearables and markerless motion capture (Boswell et al., 2023; Gilon et al., 2026; Knudson, 2011; Uhlrich et al., 2023; Werling et al., 2023), and data processing is increasingly performed on cloud-based servers such as OpenCap (Uhlrich et al., 2023) and AddBiomechanics (Werling et al., 2023). Existing visualization workflows do not scale to these settings: we recently conducted a study using OpenCap with 129 individuals performing 10 activities, and manually loading 1,290 trials for quality control in a desktop GUI is infeasible. Comparing the effects of different algorithms on a single motion is a similarly common biomechanics task that currently requires extensive manual interaction. The community therefore lacks a flexible tool for programmatic, real-time, and shareable visualization of movement data, and this gap is felt most acutely by researchers running cloud-based pipelines, large field studies, and real-time experiments.

OpenCap Visualizer addresses this gap with a Python API for batch-generating reproducible videos and figures, a browser-based viewer for collaborators to inspect 3D data via lightweight URLs, and a WebSocket interface for real-time streaming. The platform builds on the OpenCap ecosystem (Uhlrich et al., 2023) and is compatible with OpenSim models and standard

40 biomechanics data formats (Seth et al., 2018), making it applicable to both traditional marker-
41 based workflows and emerging markerless approaches using cameras (Gilon et al., 2026; Uhlrich
42 et al., 2023) or IMUs (al Borno et al., 2022; Slade et al., 2021).

43 State of the Field

44 The OpenSim GUI (Seth et al., 2018) is a widely used visualization tool in musculoskeletal
45 biomechanics and is optimized for interactive, single-trial analysis on local machines through a
46 desktop Java application; it does not expose a scripting interface for video generation and does
47 not scale to the automated workflows required for cloud-processed datasets. Mokka and the
48 underlying Biomechanical ToolKit (BTK) (Barre & Armand, 2014) target C3D-based motion
49 capture data and similarly rely on desktop interaction. Physics-oriented packages such as
50 MuJoCo (Todorov et al., 2012) include integrated viewers but are designed around simulation
51 rather than analysis of measured human movement. Web-based viewers bundled with cloud
52 platforms such as AddBiomechanics (Werling et al., 2023) are tightly coupled to a single
53 processing backend and dataset format, restricting their use as general-purpose visualization
54 layers.

55 We chose to build a new format-agnostic tool rather than extend the OpenSim GUI because
56 cloud-based processing pipelines require server-side, headless rendering that is impractical
57 inside a desktop Java application; browser-based 3D rendering enables zero-install collaboration
58 via shareable URLs; and unifying live streaming, batch video, and interactive viewing under a
59 single data model substantially reduces the friction of moving between visualization modes
60 during research.

61 Key Features

62 OpenCap Visualizer supports three complementary modes of interaction: browser-based
63 visualization, live streaming, and automated rendering via Python. It supports common
64 biomechanics data formats, including OpenSim models (.osim), motion and force files (.sto,
65 .mot), marker trajectories (.trc), JSON-based kinematics (e.g., OpenCap), and SMPL motion
66 sequences (.pkl) (Loper et al., 2015; Seth et al., 2018; Uhlrich et al., 2023).

67 1. Interactive Web-Based Visualization

68 OpenCap Visualizer provides installation-free 3D visualization directly in the browser using
69 Three.js. It supports anatomically accurate skeletal rendering, multi-subject overlays, markers
70 (.trc), ground reaction forces (.mot), and synchronized video playback. Users can interactively
71 control playback, camera views, colors, and transparency, and export high-resolution images,
72 videos, or timelapse composites for figures and presentations. Figure 1 illustrates multi-subject
73 comparison, including overlay of OpenCap Monocular (Gilon et al., 2026) and marker-based
74 motion capture. Figure 2 shows synchronized motion, marker, and ground reaction force data.
75 Figure 3 presents the timelapse rendering used to summarize dynamic movement patterns in
76 static figures.

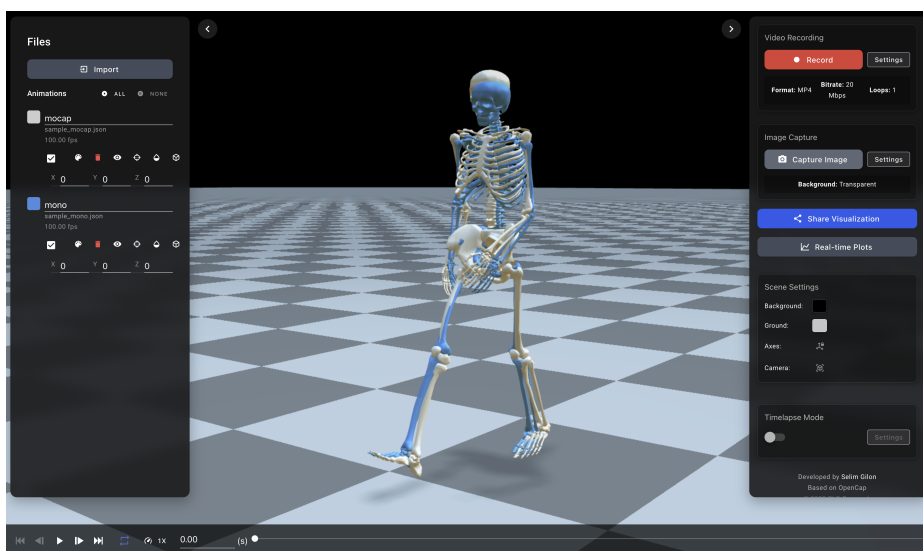


Figure 1: Example visualization of the web interface with multiple subjects.

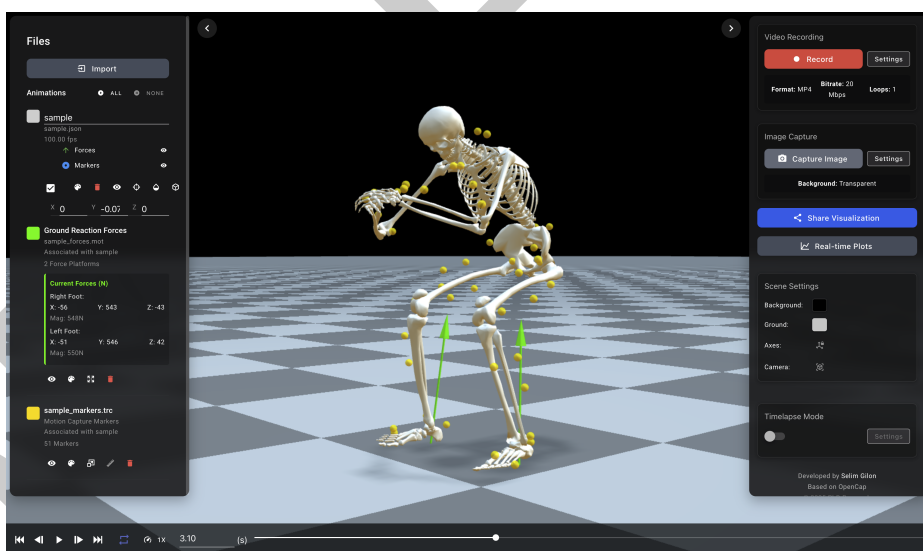


Figure 2: Example visualization of the web interface with markers and forces.

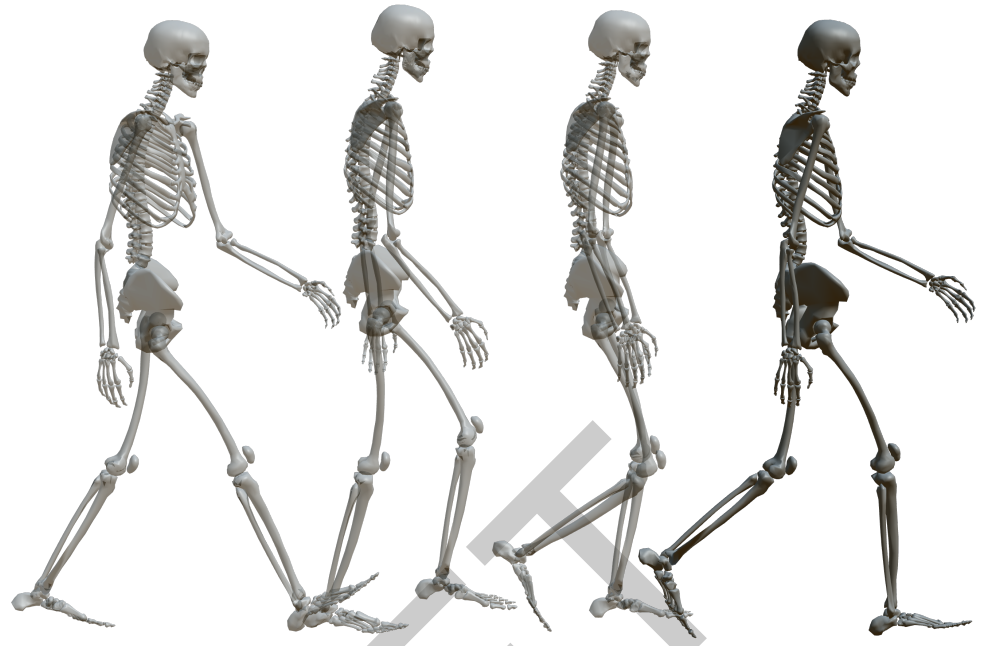


Figure 3: Example timelapse visualization generated on the visualizer.

2. Live Streaming of Kinematics

In addition to offline playback, the visualizer supports real-time streaming of OpenSim-based kinematics via a lightweight Python WebSocket server. Stream setup and frame helpers (e.g., `build_live_init_dict`, `send_live_init`, `send_live_frame`) are included in the `opencap-visualizer` pip package (PyPI). Incoming frames are incrementally rendered in the browser (Figure 4). Multiple concurrent streams (e.g., predicted vs. reference motion; Figure 1) can be displayed simultaneously. This enables real-time monitoring of inverse kinematics, model validation during data collection, and flexible visualization of results from real-time inverse kinematics pipelines such as OpenSenseRT (Slade et al., 2021). Example usage from the package includes:

```
import asyncio
import websockets
from opencap_visualizer import (
    build_live_init_dict,
    send_live_init,
    send_live_frame,
    skeleton_bodies_from_visualizer_json,
)

TEMPLATE = "subject1.json" # or any visualizer JSON with bodies

async def handler(websocket):
    meta = skeleton_bodies_from_visualizer_json(TEMPLATE)
    init = build_live_init_dict(
        [{"id": "ik", "label": "Test", "bodies": meta, "model": "LaiArnold"}],
        frame_rate=30.0,
        camera="anterior",
    )
    await send_live_init(websocket, init, mesh_load_delay=0.5)
```

```

frame_rate_hz = 30.0
frame_dt = 1.0 / frame_rate_hz

# Continuous real-time stream.
# Finite demo: break after N frames or set streaming = False.
streaming = True
t = 0.0
while streaming:
    # Body pose (example): rotation in rad, translation in model units.
    # Sample rotation: [0.0, 0.0, 0.0]; sample translation: [0.0, 1.0, 0.0]
    rot_x = 0.0
    rot_y = 0.0
    rot_z = 0.0
    trans_x = 0.0
    trans_y = 1.0
    trans_z = 0.0

    bodies = {}
    for body_name in meta:
        bodies[body_name] = {
            "rotation": [rot_x, rot_y, rot_z],
            "translation": [trans_x, trans_y, trans_z],
        }

    await send_live_frame(websocket, {"ik": {"time": t, "bodies": bodies}})
    t += frame_dt
    await asyncio.sleep(frame_dt)

async def main():
    async with websockets.serve(handler, "0.0.0.0", 876):
        await asyncio.Future()

asyncio.run(main())

```

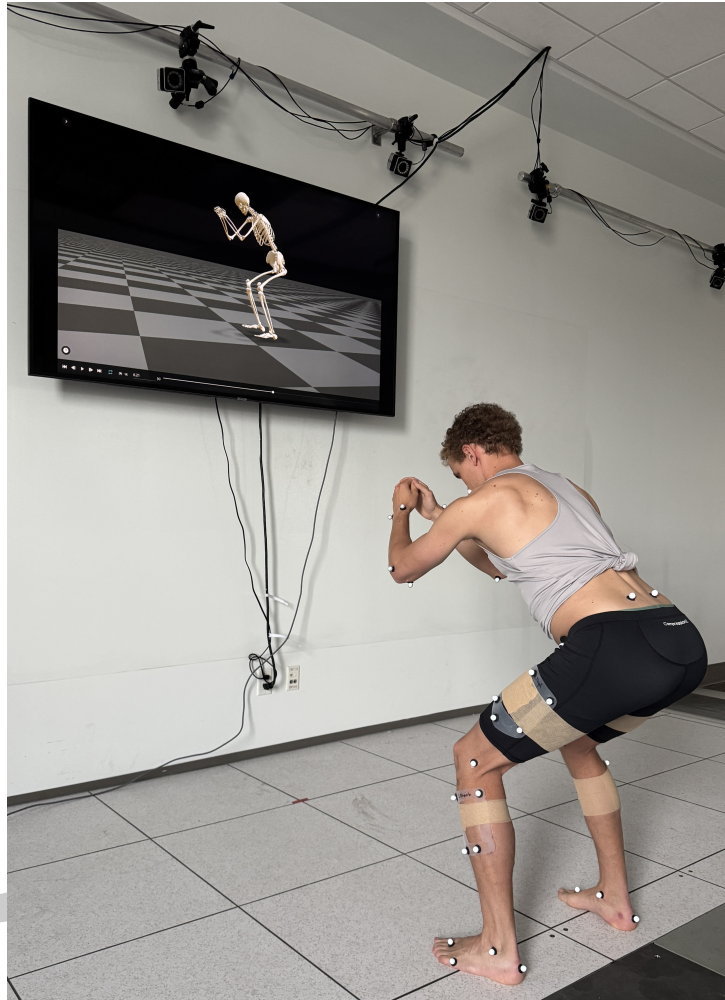


Figure 4: Example visualization of OpenSim kinematics in livestream (real-time).

3. Python API for Automated Video Creation

The `opencap-visualizer` Python package (<https://pypi.org/project/opencap-visualizer>) enables fully programmatic video generation for integration into automated pipelines and headless servers. Users can batch-render videos with configurable camera views, subject overlays, colors, and looping behavior from standard OpenCap and OpenSim inputs, eliminating the need for manual GUI interaction. Example commands for video rendering include:

```
import opencap_visualizer as ocv

# Single-subject OpenCap data
ocv.create_video("subj.json", "output.mp4", camera="anterior")
# Multi-subject comparison from OpenCap jsons
ocv.create_video(["subj1.json", "subj2.json"], "compare.mp4", colors=["red", "blue"])
# Multiple synchronized data modalities
paths = ["model.osim", "motion.mot", "markers.trc", "forces.mot"]
ocv.create_video(paths, "experimental_visualization.mp4")
```


Software Design

OpenCap Visualizer is structured as a thin web frontend, a stateless cloud service, and a Python client package, with all three layers sharing a single intermediate JSON representation of bodies, motions, markers, and forces. Centering the architecture on this representation rather than a particular file format was a deliberate trade-off: it adds an explicit conversion step on the server, but decouples the rendering pipeline from OpenSim's C++ runtime and lets the same browser viewer drive interactive playback, headless video export through a Puppeteer-controlled Chromium instance, and live streaming. This shared data model is what makes both batch generation on cloud servers and real-time streaming from local inverse kinematics pipelines fit into a single codebase.

We chose Vue.js and Three.js over a more specialized rendering toolkit to keep the dependency surface compatible with shared hosting, embeddable iframes, and mobile browsers, broadening accessibility for collaborators who do not run dedicated biomechanics software. The Python package wraps a headless browser to reproduce the exact frontend renderer, ensuring that automatically generated videos are visually identical to interactive playback—an important property for paper figures and reproducibility, but difficult to achieve when the offline renderer is a separate program. The live-streaming server is intentionally minimal: it forwards body transforms over WebSockets without owning model state, so users can plug in any inverse kinematics backend (e.g., OpenSenseRT (Slade et al., 2021)) without modifying the visualizer.

The frontend is built with Vue.js and Three.js, with Vuetify providing UI components. The deployed web application includes cloud-based services for URL-based sharing and server-side conversion of OpenSim models and motion files into the browser-compatible JSON representation. For local and automated workflows, the lightweight Python package enables headless video generation, while a companion Python module supports real-time kinematics streaming via WebSockets.

Applications

OpenCap Visualizer supports high-throughput biomechanics workflows where manual, GUI-based visualization is impractical.

- **Algorithm development:** The ability to programmatically create videos with multiple models and data modalities allows researchers to quickly visualize comparisons when developing new algorithms or performing validation studies. This was previously a laborious task requiring extensive GUI interaction.
- **Quality control for large datasets:** A single video can be compiled of every motion trial in a dataset enabling rapid quality control after large data collections. This is an essential step for both lab-based and out-of-lab biomechanics experiments and was cumbersome with prior GUI-based workflows.
- **Reproducible figures for publications:** Timelapse rendering allows dynamic motion to be represented in static, publication-quality figures, facilitating clear qualitative comparisons in print.
- **Real-time biofeedback:** Real-time streaming enables real-time biofeedback studies using inverse kinematics from pipelines like OpenSenseRT (Slade et al., 2021).
- **Education and clinical documentation:** Browser-based visualization removes installation barriers, enabling interactive teaching materials and standardized video generation for documenting patient movement and intervention outcomes.

Research Impact Statement

OpenCap Visualizer is integrated into the OpenCap (Uhlrich et al., 2023) processing pipeline and serves as the default visualization tool for OpenCap Monocular (Gilon et al., 2026). The Python package is published on PyPI (<https://pypi.org/project/opencap-visualizer>) and has been used by our group to run a parameter optimization across several hundred trials to visualize the resulting motions from OpenCap Monocular, with both the qualitative figures and the supplementary motion videos in (Gilon et al., 2026) generated directly with the tool. It has also been used to develop and illustrate a foundation model of gait kinematics (Magruder et al., 2026). The browser viewer has further supported peer collaboration on OpenCap projects through shareable URLs. The codebase is released open-source under the Apache License 2.0 on GitHub with public issue tracking, automated PyPI releases, and worked examples in the repository. Near-term uptake is anticipated by groups working with markerless motion capture, real-time biofeedback, and large-scale field studies, where the absence of a scriptable, browser-based visualizer has been a recurring bottleneck.

AI Usage Disclosure

Generative AI assistants (GitHub Copilot and Anthropic Claude) were used during software development to draft boilerplate code, write documentation, and propose refactorings, and during paper writing to polish prose and tighten phrasing. All AI-generated code was reviewed, tested, and edited by the authors before being merged.

Usage Summary

The web visualizer is available at <https://www.visualizer.opencap.ai>. The Python package, *opencap-visualizer*, can be installed via pip at <https://pypi.org/project/opencap-visualizer>. The open-source code and example Python scripts are available at <https://github.com/Seeeeeeyo/opencap-visualizer>.

References

- al Borno, M., O'Day, J., Ibarra, V., Dunne, J., Seth, A., Habib, A., Ong, C., Uhlrich, S., & Delp, S. (2022). OpenSense: An open-source toolbox for inertial-measurement-unit-based measurement of lower extremity kinematics over long durations. *Journal of NeuroEngineering and Rehabilitation*, 19. <https://doi.org/10.1186/s12984-022-01001-x>
- Barre, A., & Armand, S. (2014). Biomechanical ToolKit: Open-source framework to visualize and process biomechanical data. *Computer Methods and Programs in Biomedicine*, 114(1), 80–87. <https://doi.org/10.1016/j.cmpb.2014.01.012>
- Boswell, M. A., Kidziński, Ł., Hicks, J. L., Uhlrich, S. D., Falisse, A., & Delp, S. L. (2023). Smartphone videos of the sit-to-stand test predict osteoarthritis and health outcomes in a nationwide study. *Npj Digital Medicine*, 6(1), 32. <https://doi.org/10.1038/s41746-023-00775-1>
- Gilon, S., Miller, E. Y., & Uhlrich, S. D. (2026). *OpenCap monocular: 3D human kinematics and musculoskeletal dynamics from a single smartphone video*. <https://doi.org/10.48550/arXiv.2603.24733>
- Knudson, D. V. (2011). Authorship and sampling practice in selected biomechanics and sports science journals. *Perceptual and Motor Skills*, 112(3), 838–844. <https://doi.org/10.2466/17.PMS.112.3.838-844>
- Loper, M., Mahmood, N., Romero, J., Pons-Moll, G., & Black, M. J. (2015). SMPL: A

- 180 skinned multi-person linear model. *ACM Trans. Graph.*, 34(6). [https://doi.org/10.1145/](https://doi.org/10.1145/2816795.2818013)
181 [2816795.2818013](https://doi.org/10.1145/2816795.2818013)
- 182 Magruder, R. D., Gilon, S., Falisse, A., & Uhlich, S. D. (2026). GaitEncoder: A foundation
183 model of gait kinematics for diverse clinical applications and pathologies. *medRxiv*. <https://doi.org/10.64898/2026.07.07.26357479>
184
- 185 Seth, A., Hicks, J. L., Uchida, T. K., Habib, A., Dembia, C. L., Dunne, J. J., & al., et.
186 (2018). OpenSim: Simulating musculoskeletal dynamics and neuromuscular control to
187 study human and animal movement. *PLoS Computational Biology*, 14(7), e1006223.
188 <https://doi.org/10.1371/journal.pcbi.1006223>
- 189 Slade, P., Habib, A., Hicks, J., & Delp, S. (2021). An open-source and wearable system for
190 measuring 3D human motion in real-time. *IEEE Transactions on Biomedical Engineering*,
191 *PP*, 1–1. <https://doi.org/10.1109/TBME.2021.3103201>
- 192 Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control.
193 *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 5026–5033.
194 <https://doi.org/10.1109/IROS.2012.6386109>
- 195 Uhlich, S. D., Falisse, A., Kidziński, Ł., Muccini, J., Ko, M., Chaudhari, A. S., Hicks, J. L., &
196 Delp, S. L. (2023). OpenCap: Human movement dynamics from smartphone videos. *PLOS*
197 *Computational Biology*, 19(10), 1–26. <https://doi.org/10.1371/journal.pcbi.1011462>
- 198 Werling, K., Bianco, N. A., Raitor, M., Stingel, J., Hicks, J. L., Collins, S. H., Delp, S. L., &
199 Liu, C. K. (2023). AddBiomechanics: Automating model scaling, inverse kinematics, and
200 inverse dynamics from human motion data through sequential optimization. *PLOS ONE*,
201 *18*(11), 1–25. <https://doi.org/10.1371/journal.pone.0295152>